

1 List Comprehension – Numbers & Strings

Title: *Even-Odd Formatter & List Utilities*

Overview:

Given a list of integers, create utilities using **list comprehensions only** to extract information, transform values, and generate new collections. Input order must be preserved unless explicitly stated otherwise.

Functionalities

#	Function Signature	Description
1	<code>format_numbers(nums: List[int]) -> Tuple[List[int], List[str]]</code>	Returns a tuple (<code>evens_sq</code> , <code>odds_label</code>): <code>evens_sq</code> contains squares of even numbers, <code>odds_label</code> contains the string "odd" for each odd number.
2	<code>filter_even(nums: List[int]) -> List[int]</code>	Returns a list of even numbers from <code>nums</code> .
3	<code>sum_of_squares(nums: List[int]) -> int</code>	Computes the sum of squares of all numbers in <code>nums</code> .
4	<code>concat_strings(strs: List[str]) -> str</code>	Concatenates strings in <code>strs</code> into a single string, separated by a single space.
5	<code>range_from_min_max(nums: List[int]) -> List[int]</code>	Generates a list of integers from the minimum to the maximum value in <code>nums</code> (inclusive). Returns an empty list if <code>nums</code> is empty.

Test Cases

#	Input (nums / strs)	Expected Output (1-5)
1	<code>nums=[1,2,3,4,5] strs=["a","b","c"]</code>	<code>1</code> <code>((4,16), ['odd', 'odd', 'odd'])</code> <code>2</code> <code>[2,4]</code> <code>3</code> <code>55</code> <code>4</code> <code>"a b c"</code> <code>5</code> <code>[1,2,3,4,5]</code>
2	<code>nums=[] strs=[]</code>	<code>1</code> <code>([], [])</code> <code>2</code> <code>[]</code> <code>3</code> <code>0</code> <code>4</code> <code>""</code> <code>5</code> <code>[]</code>
3	<code>nums=[0,-2,-3] strs=["x"]</code>	<code>1</code> <code>((0,4), ['odd'])</code> <code>2</code> <code>[0,-2]</code> <code>3</code> <code>13</code> <code>4</code> <code>"x"</code> <code>5</code> <code>[-3,-2,-1,0]</code>
4	<code>nums=[7,8,9] strs=["hello","world"]</code>	<code>1</code> <code>((64), ['odd', 'odd'])</code> <code>2</code> <code>[8]</code> <code>3</code> <code>194</code> <code>4</code> <code>"hello world"</code> <code>5</code> <code>[7,8,9]</code>
5	<code>nums=[5] strs=["solo"]</code>	<code>1</code> <code>([], ['odd'])</code> <code>2</code> <code>[]</code> <code>3</code> <code>25</code> <code>4</code> <code>"solo"</code> <code>5</code> <code>[5]</code>
6	<code>nums=[-4,-3,-2,-1,0,1] strs=["x","y"]</code>	<code>1</code> <code>((16,4,0), ['odd', 'odd', 'odd'])</code> <code>2</code> <code>[-4,-2,0]</code> <code>3</code> <code>31</code> <code>4</code> <code>"x y"</code> <code>5</code> <code>[-4,-3,-2,-1,0,1]</code>
7	<code>nums=[1000] strs=["large"]</code>	<code>1</code> <code>((1000000), [])</code> <code>2</code> <code>[1000]</code> <code>3</code> <code>1000000</code> <code>4</code> <code>"large"</code> <code>5</code> <code>[1000]</code>
8	<code>nums=[0,0,0] strs=[""]</code>	<code>1</code> <code>((0,0,0), [])</code> <code>2</code> <code>[0,0,0]</code> <code>3</code> <code>0</code> <code>4</code> <code>""</code> <code>5</code> <code>[0]</code>

2 String Operations

Title: *Comprehensive String Toolkit*

Overview:

Implement utilities for string manipulation, including searching, transformation, counting, and validation. All functions must be pure (no side effects) and support Unicode characters.

Functionalities

#	Function Signature	Description
1	<code>palindromic_substrings(s: str) -> List[str]</code>	Returns all unique palindrome substrings of length ≥ 3 , sorted alphabetically.
2	<code>reverse_words(s: str) -> str</code>	Reverses the order of words in <code>s</code> while preserving each word. Words are separated by whitespace (including multiple spaces).

#	Function Signature	Description
3	<code>count_vowels(s: str) -> int</code>	Counts vowels (<code>a,e,i,o,u</code> , case-insensitive) in <code>s</code> .
4	<code>is_anagram(s1: str, s2: str) -> bool</code>	Returns <code>True</code> if <code>s1</code> and <code>s2</code> contain the same letters (ignoring case and non-alphabetic characters).
5	<code>title_case(s: str) -> str</code>	Capitalizes each word (first letter uppercase, rest lowercase).

Test Cases

#	Input (<code>s</code> / <code>s1,s2</code>)	Expected Output (1-5)
1	<code>s="ababa"</code>	<code>1</code> <code>['aba','ababa','bab']</code> <code>2</code> <code>"ababa"</code> <code>3</code> <code>3</code> <code>4</code> <code>is_anagram("Listen","Silent") -> True</code> <code>5</code> <code>"Ababa"</code>
2	<code>s="Madam, I'm Adam"</code>	<code>1</code> <code>[' ada ','Madam','ada']</code> <code>2</code> <code>"Adam I'm Madam,"</code> <code>3</code> <code>5</code> <code>4</code> <code>is_anagram("Hello!","Olelh") -> True</code> <code>5</code> <code>"Madam, I'm Adam"</code>
3	<code>s="abc"</code>	<code>1</code> <code>[]</code> <code>2</code> <code>"abc"</code> <code>3</code> <code>1</code> <code>4</code> <code>is_anagram("abc","def") -> False</code> <code>5</code> <code>"Abc"</code>
4	<code>s="racecar"</code>	<code>1</code> <code>['aceca','cec','racecar']</code> <code>2</code> <code>"racecar"</code> <code>3</code> <code>3</code> <code>4</code> <code>is_anagram("Dormitory","Dirty room") -> True</code> <code>5</code> <code>"Racecar"</code>
5	<code>s=" multiple spaces "</code>	<code>1</code> <code>[]</code> <code>2</code> <code>"spaces multiple"</code> <code>3</code> <code>6</code> <code>4</code> <code>is_anagram("a!b@c#", "C B A") -> True</code> <code>5</code> <code>" Multiple Spaces "</code>
6	<code>s="αβγγα"</code>	<code>1</code> <code>['αγγα']</code> <code>2</code> <code>"αβγγα"</code> <code>3</code> <code>2</code> <code>4</code> <code>is_anagram("αβγ","γβα") -> True</code> <code>5</code> <code>"Αβγγα"</code>
7	<code>s=""</code>	<code>1</code> <code>[]</code> <code>2</code> <code>""</code> <code>3</code> <code>0</code> <code>4</code> <code>is_anagram("", "") -> True</code> <code>5</code> <code>""</code>
8	<code>s="a a a"</code>	<code>1</code> <code>[]</code> <code>2</code> <code>"a a a"</code> <code>3</code> <code>3</code> <code>4</code> <code>is_anagram("a a a","aaa") -> True</code> <code>5</code> <code>"A A A"</code>

3 Dictionaries & Dictionary Comprehension

Title: *Dictionary Mastery Suite*

Overview:

Create utilities to manipulate dictionaries using **dictionary comprehensions**, including frequency counting, inversion, merging, filtering, and sorting. Functions must work with generic key/value types where applicable.

Functionalities

#	Function Signature	Description
1	<code>char_freq(text: str) -> Dict[str, int]</code>	Maps each lowercase alphabetic character to its frequency using a one-line dictionary comprehension. Non-alphabetic characters are ignored.
2	<code>invert_dict(d: Dict[Any, Any]) -> Dict[Any, List[Any]]</code>	Maps each value to a list of keys that had that value.
3	<code>merge_dicts(a: Dict[Any, int], b: Dict[Any, int]) -> Dict[Any, int]</code>	Merges two dictionaries, summing values for common keys; keeps missing keys as-is.
4	<code>filter_by_value(d: Dict[Any, int], threshold: int) -> Dict[Any, int]</code>	Returns items where value $\geq$ <code>threshold</code> .
5	<code>sort_by_key(d: Dict[Any, Any]) -> List[Tuple[Any, Any]]</code>	Returns a list of (<code>key</code> , <code>value</code>) tuples sorted by key (ascending).

Test Cases

#	Input (<code>text</code> / <code>a,b,threshold</code>)	Expected Output (1-5)
1	<code>text="Hello World!" a={'x':1,'y':2} b={'y':3,'z':4} threshold=2</code>	<code>1</code> <code>{'h':1,'e':1,'l':3,'o':2,'w':1,'r':1,'d':1}</code> <code>2</code> <code>{1:['x'],2:['y'],3:['y'],4:['z']}</code> <code>3</code> <code>{'x':1,'y':5,'z':4}</code> <code>4</code> <code>{'y':5,'z':4}</code> <code>5</code> <code>[('x',1), ('y',5), ('z',4)]</code>

#	Input (text / a,b,threshold)	Expected Output (1-5)
2	text="12345" a={} b={'a':10} threshold=5	1 {} 2 {10:['a']} 3 {'a':10} 4 {'a':10} 5 [('a',10)]
3	text="AaBbCc" a={'p':0} b={} threshold=1	1 {'a':2,'b':2,'c':2} 2 {0:['p']} 3 {'p':0} 4 {} 5 [('p',0)]
4	text="" a={'k':5,'l':3} b={'k':-2,'m':7} threshold=4	1 {} 2 {5:['k'],3:['l'],-2:['k'],7:['m']} 3 {'k':3,'l':3,'m':7} 4 {'k':3,'m':7} 5 [('k',3),('l',3),('m',7)]
5	text="aAa" a={'x':10,'y':20} b={'x':-5,'z':30} threshold=15	1 {'a':3} 2 {10:['x'],20:['y'],-5:['x'],30:['z']} 3 {'x':5,'y':20,'z':30} 4 {'y':20,'z':30} 5 [('x',5),('y',20),('z',30)]
6	text="αβγ" a={'x':1} b={'x':1} threshold=2	1 {'α':1,'β':1,'γ':1} 2 {1:['x','x']} 3 {'x':2} 4 {'x':2} 5 [('x',2)]
7	text="x123x" a={'a':1,'b':1} b={'c':1} threshold=2	1 {'x':2} 2 {1:['a','b','c']} 3 {'a':1,'b':1,'c':1} 4 {} 5 [('a',1),('b',1),('c',1)]

4 Nested Dictionaries

Title: *Student-Grades Analytics*

Overview:

A school stores results as a nested dictionary: `student_id -> {subject -> score}`. Build utilities to compute per-student statistics, per-subject aggregates, and modify the data structure safely.

Functionalities

#	Function Signature	Description
1	<code>average_grades(data: Dict[str, Dict[str, float]]) -> Dict[str, float]</code>	Returns each student's average score (rounded to 2 decimal places). Empty subject list -> <code>0.0</code> .
2	<code>max_grade_per_student(data: Dict[str, Dict[str, float]]) -> Dict[str, Tuple[str, float]]</code>	Returns <code>(subject, score)</code> of the highest-scoring subject per student. No subjects -> <code>('',0.0)</code> .
3	<code>subject_wise_average(data: Dict[str, Dict[str, float]]) -> Dict[str, float]</code>	Computes the average score per subject across all students (rounded to 2 decimal places).
4	<code>flatten_nested(data: Dict[str, Dict[str, float]]) -> List[Tuple[str, str, float]]</code>	Returns a list of <code>(student_id, subject, score)</code> tuples.
5	<code>add_student_record(data: Dict[str, Dict[str, float]], sid: str, record: Dict[str, float]) -> None</code>	Inserts or updates a student's record with <code>record</code> .

Test Cases

#	Input (data / sid,record)	Expected Output (1-5)
1	data={'s1':{'math':80,'eng':90},'s2':{'math':70}} sid='s3' record={'eng':85,'sci':78}	1 {'s1':85.0,'s2':70.0} 2 {'s1':('eng',90.0),'s2':('math',70.0)} 3 {'math':75.0,'eng':90.0,'sci':78.0} 4 [('s1','math',80.0),('s1','eng',90.0),('s2','math',70.0)] 5 data['s3']= {'eng':85.0,'sci':78.0}
2	data={} sid='new' record={}	1 {} 2 {} 3 {} 4 [] 5 data['new']={}
3	data={'s3':{}} sid='s3' record={'bio':92}	1 {'s3':0.0} 2 {'s3':('',0.0)} 3 {'bio':92.0} 4 [] 5 data['s3']= {'bio':92.0}
4	data={'a':{'x':95,'y':85,'z':75}} sid='b' record={'x':60}	1 {'a':85.0} 2 {'a':('x',95.0)} 3 {'x':95.0,'y':85.0,'z':75.0} 4 [('a','x',95.0),('a','y',85.0),('a','z',75.0)] 5 data['b']={'x':60.0}

#	Input (data / sid,record)	Expected Output (1-5)
5	data={'p':{'m':50}} sid='p' record={'m':70,'n':80}	1 {'p':50.0} 2 {'p':('m',50.0)} 3 {'m':70.0,'n':80.0} 4 [('p','m',50.0)] 5 data['p']={'m':70.0,'n':80.0}
6	data={'s1': {'math':100,'eng':0}} sid='s2' record={'math':50.5}	1 {'s1':50.0} 2 {'s1':('math',100.0)} 3 {'math':100.0,'eng':0.0} 4 [('s1','math',100.0),('s1','eng',0.0)] 5 data['s2']={'math':50.5}

5 Basic One-Class OOP – Operations on Dictionary of Objects

Title: *Inventory Management Toolkit*

Overview:

Design a simple `Item` class to store product details and implement a manager for a dictionary of `Item` objects, providing CRUD-style operations and analytics.

Functionalities

#	Function / Class Signature	Description
1	<code>class Item</code> <code>__init__(self, name: str, price: float, qty: int)</code>	Stores <code>name</code> , <code>price</code> , and <code>qty</code> .
2	<code>update_inventory(inv: Dict[str, Item], code: str, add_qty: int) -> float</code>	Adjusts quantity for <code>code</code> (ignores missing keys) and returns total inventory value (<code>price * qty</code> sum).
3	<code>total_quantity(inv: Dict[str, Item]) -> int</code>	Sums <code>qty</code> of all items.
4	<code>most_valuable_item(inv: Dict[str, Item]) -> Tuple[str, float]</code>	Returns (<code>code</code> , <code>value</code>) of the item with the highest <code>price * qty</code> . Ties return any valid item; empty dict -> <code>('',0.0)</code> .
5	<code>add_item(inv: Dict[str, Item], code: str, item: Item) -> None</code>	Inserts or overwrites an entry with <code>item</code> .
6	<code>remove_item(inv: Dict[str, Item], code: str) -> bool</code>	Deletes <code>code</code> from the dict; returns <code>True</code> if deleted, <code>False</code> otherwise.

Test Cases

#	Initial inv	Operation	Arguments	Expected Result	Post-state Check
1	<code>{'A':Item('Pen',2.5,10), 'B':Item('Book',5.0,4)}</code>	<code>update_inventory</code>	<code>('A',5)</code>	57.5	<code>inv['A'].qty == 15</code>
2	Same as 1	<code>update_inventory</code>	<code>('C',3)</code>	57.5	Dict unchanged
3	Same as 1	<code>total_quantity</code>	–	14	–
4	Same as 1	<code>most_valuable_item</code>	–	<code>('A',37.5)</code>	–
5	Same as 1	<code>add_item</code>	<code>('D',Item('Ruler',1.2,8))</code>	<code>None</code>	<code>inv['D'].name == 'Ruler'</code>
6	Same as 1	<code>remove_item</code>	<code>('B')</code>	<code>True</code>	<code>'B' not in inv</code>
7	Same as 1	<code>remove_item</code>	<code>('X')</code>	<code>False</code>	Dict unchanged
8	<code>{}</code>	<code>most_valuable_item</code>	–	<code>('',0.0)</code>	–
9	<code>{'X':Item('Zero',10.0,0)}</code>	<code>total_quantity</code>	–	0	–
10	<code>{'Y':Item('Single',3.3,1)}</code>	<code>update_inventory</code>	<code>('Y',-1)</code>	0.0	<code>inv['Y'].qty == 0</code>
11	<code>{'Z':Item('Test',0.0,10)}</code>	<code>most_valuable_item</code>	–	<code>('Z',0.0)</code>	–

6 Abstract Base Class & Inheritance – Operations on Dictionaries of Objects

Title: Shape Hierarchy & Geometry Engine

Overview:

Create an abstract base class `Shape` with concrete subclasses `Circle` and `Rectangle`. Implement utilities to perform calculations, scaling, and descriptive operations on a dictionary of `Shape` objects. Areas should be rounded to 2 decimal places in output.

Functionalities

#	Function / Class Signature	Description
1	<pre>class Shape(ABC) @abstractmethod def area(self) -> float</pre>	Abstract base class with an <code>area</code> method.
2	<pre>class Circle(Shape) __init__(self, radius: float)</pre>	Implements <code>area = $\pi \cdot r^2$</code> .
3	<pre>class Rectangle(Shape) __init__(self, w: float, h: float)</pre>	Implements <code>area = w.h</code> .
4	<pre>calc_areas(shapes: Dict[str, Shape]) -> Dict[str, float]</pre>	Returns <code>{id: area}</code> for all shapes (rounded to 2 decimal places).
5	<pre>total_area(shapes: Dict[str, Shape]) -> float</pre>	Sums all shape areas (rounded to 2 decimal places).
6	<pre>largest_shape(shapes: Dict[str, Shape]) -> Tuple[str, float]</pre>	Returns <code>(id, area)</code> of the shape with the largest area (rounded to 2 decimal places); empty dict -> <code>('', 0.0)</code> .
7	<pre>scale_shape(shapes: Dict[str, Shape], factor: float) -> None</pre>	Multiplies all numeric dimensions (radius, width, height) by <code>factor</code> .
8	<pre>describe_shape(shapes: Dict[str, Shape]) -> List[str]</pre>	Returns sorted list of strings like <code>"Circle(id=c1, radius=2.00, area=12.57)"</code> or <code>"Rectangle(id=r1, width=2.00, height=3.00, area=6.00)"</code> with values rounded to 2 decimal places.
9	<pre>filter_by_type(shapes: Dict[str, Shape], typ: type) -> Dict[str, Shape]</pre>	Returns a dictionary containing only instances of <code>typ</code> (<code>Circle</code> or <code>Rectangle</code>).

Test Cases

#	Shapes Dict	Expected Output (4-9)
1	<pre>{'c1': Circle(1), 'r1': Rectangle(2,3)}</pre>	4 <code>{'c1':3.14,'r1':6.0}</code> 5 9.14 6 <code>('r1',6.0)</code> 7 After scaling by 2: <code>c1.radius=2, r1.w=4, r1.h=6</code> 8 <code>["Circle(id=c1, radius=2.00, area=12.57)", "Rectangle(id=r1, width=4.00, height=6.00, area=24.00)"]</code> 9 <code>{'c1':Circle(2)}</code> (Circle)
2	<pre>{}</pre>	4 <code>{}</code> 5 0.0 6 <code>('',0.0)</code> 7 No error 8 <code>[]</code> 9 <code>{}</code> (Circle)
3	<pre>{'c2': Circle(0), 'r2': Rectangle(5,0)}</pre>	4 <code>{'c2':0.0,'r2':0.0}</code> 5 0.0 6 <code>('c2',0.0)</code> 7 Scaling by 3: all zeros 8 <code>["Circle(id=c2, radius=0.00, area=0.00)", "Rectangle(id=r2, width=5.00, height=0.00, area=0.00)"]</code> 9 <code>{'c2':Circle(0)}</code> (Circle)
4	<pre>{'c3': Circle(2.5)}</pre>	4 <code>{'c3':19.63}</code> 5 19.63 6 <code>('c3',19.63)</code> 7 Scaling by 0.5: <code>c3.radius=1.25</code> 8 <code>["Circle(id=c3, radius=1.25, area=4.91)"]</code> 9 <code>{'c3':Circle(1.25)}</code> (Circle)
5	<pre>{'r3': Rectangle(4,5), 'r4': Rectangle(3,3)}</pre>	4 <code>{'r3':20.0,'r4':9.0}</code> 5 29.0 6 <code>('r3',20.0)</code> 7 Scaling by -1: negative dimensions 8 <code>["Rectangle(id=r3, width=-4.00, height=-5.00, area=20.00)", "Rectangle(id=r4, width=-3.00, height=-3.00, area=9.00)"]</code> 9 <code>{}</code> (Circle)
6	<pre>{'c4': Circle(-1)}</pre>	4 <code>{'c4':3.14}</code> 5 3.14 6 <code>('c4',3.14)</code> 7 Scaling by 2: <code>c4.radius=-2</code> 8 <code>["Circle(id=c4, radius=-2.00, area=12.57)"]</code> 9 <code>{'c4':Circle(-2)}</code> (Circle)

7 File I/O – Basic Read/Write of List-Based Data

Title: *Plain-Text List Persistence*

Overview:

Implement a library to store a list of strings to a file, read it back, and provide utilities for appending, counting lines, and cleaning up empty files. All functions must handle file operations robustly, support Unicode characters, and avoid raising exceptions for missing files unless specified.

Functionalities

#	Function Signature	Description
1	<code>save_list(path: str, items: List[str]) -> None</code>	Overwrites <code>path</code> with each element on its own line. Creates the file if it does not exist.
2	<code>load_list(path: str) -> List[str]</code>	Returns a list of lines (newlines stripped). Returns an empty list if the file is missing.
3	<code>append_to_list(path: str, items: List[str]) -> None</code>	Appends <code>items</code> to <code>path</code> , each on a new line. Creates the file if it does not exist.
4	<code>count_lines(path: str) -> int</code>	Returns the number of lines in the file. Returns <code>0</code> if the file is missing.
5	<code>delete_file_if_empty(path: str) -> bool</code>	Deletes the file if its size is zero bytes; returns <code>True</code> if deleted, <code>False</code> if the file is non-empty or missing.

Test Cases

#	Operations (in order)	Expected File Content / Return Values
1	<code>save_list('tmp.txt', ["apple","banana","cherry"]) → load_list('tmp.txt')</code>	<code>["apple","banana","cherry"]</code>
2	<code>save_list('tmp.txt', []) → load_list('tmp.txt')</code>	<code>[]</code>
3	<code>save_list('tmp.txt', ["line1"]) → append_to_list('tmp.txt', ["line2","line3"]) → load_list('tmp.txt')</code>	<code>["line1","line2","line3"]</code>
4	After previous step, <code>count_lines('tmp.txt')</code>	<code>3</code>
5	<code>save_list('empty.txt', []) → delete_file_if_empty('empty.txt')</code>	<code>True</code> , file no longer exists
6	<code>delete_file_if_empty('nonexistent.txt')</code>	<code>False</code>
7	<code>count_lines('nonexistent.txt')</code>	<code>0</code>
8	<code>append_to_list('new.txt', [" spaced "]) → load_list('new.txt')</code>	<code>[" spaced "]</code>
9	<code>save_list('unicode.txt', ["αβγ","こんにちは"]) → load_list('unicode.txt')</code>	<code>["αβγ","こんにちは"]</code>
10	<code>save_list('spaces.txt', [" ",""]) → load_list('spaces.txt')</code>	<code>[" ",""]</code>

8 File I/O + Regex – Pattern Matching

Title: *Log Generator & Error Extractor*

Overview:

Write a script to generate a deterministic log file containing 100 lines with timestamps, log levels (`INFO`, `WARN`, `ERROR`), and messages. Implement utilities to extract information using regular expressions. The log format is `"[LEVEL] YYYY-MM-DD HH:MM:SS - Message"`. Functions must handle missing files gracefully.

Functionalities

#	Function Signature	Description
---	--------------------	-------------

#	Function Signature	Description
1	<code>generate_logs(output_path: str, seed: int = 0) -> None</code>	Generates 100 log lines of format "[LEVEL] YYYY-MM-DD HH:MM:SS - Message". Messages are "Message 0" to "Message 19", chosen deterministically using <code>random.seed(seed)</code> . Timestamps range from 2025-01-01 to 2025-12-31. Creates or overwrites the file.
2	<code>extract_errors(path: str) -> List[str]</code>	Returns the message part of all <code>ERROR</code> level lines, preserving order. Returns empty list for missing files.
3	<code>count_errors_by_level(path: str) -> Dict[str, int]</code>	Returns {'INFO': x, 'WARN': y, 'ERROR': z} with counts of each log level. Returns {'INFO': 0, 'WARN': 0, 'ERROR': 0} for missing files.
4	<code>extract_dates(path: str, level: str None = None) -> List[str]</code>	Returns YYYY-MM-DD dates for all lines, optionally filtered by <code>level</code> . Returns empty list for missing files.
5	<code>filter_by_message_pattern(path: str, pattern: str) -> List[Tuple[str, str]]</code>	Returns (level, message) tuples where the message matches the regex <code>pattern</code> (case-insensitive), preserving order. Returns empty list for missing files or invalid patterns.

Test Cases

#	Seed for <code>generate_logs</code>	Expected Output (2-5)
1	0	2 List of 34 <code>ERROR</code> messages (per seed) 3 {'INFO': 34, 'WARN': 32, 'ERROR': 34} 4 34 dates for <code>level='ERROR'</code> 5 Pattern <code>".*"</code> returns 100 (level, message) tuples
2	42	2 List of 31 <code>ERROR</code> messages (per seed) 3 {'INFO': 31, 'WARN': 38, 'ERROR': 31} 4 38 dates for <code>level='WARN'</code> 5 Pattern <code>"Message 1.*"</code> returns matching (level, message) tuples
3	999	2 List of 32 <code>ERROR</code> messages (per seed) 3 {'INFO': 33, 'WARN': 35, 'ERROR': 32} 4 100 dates for <code>level=None</code> 5 Pattern <code>"Message [0-9]+"</code> returns matching (level, message) tuples
4	Non-existent file	2 [] 3 {'INFO': 0, 'WARN': 0, 'ERROR': 0} 4 [] 5 []
5	123	2 List of 34 <code>ERROR</code> messages (per seed) 3 {'INFO': 33, 'WARN': 33, 'ERROR': 34} 4 33 dates for <code>level='INFO'</code> 5 Pattern <code>".*fail.*"</code> returns [] (no messages contain "fail")
6	0	2 List of 34 <code>ERROR</code> messages (per seed) 3 {'INFO': 34, 'WARN': 32, 'ERROR': 34} 4 32 dates for <code>level='WARN'</code> 5 Pattern <code>"Nonexistent"</code> returns []

9 List & Dictionaries + Functional Programming

Title: Functional Data Transformers

Overview:

Using functional programming tools (`map`, `filter`, `reduce`, `zip`) from the `functools` and built-in Python libraries, implement utilities to transform collections without explicit `for` loops. Emphasize immutability, composability, and functional purity. All functions must support Unicode strings.

Functionalities

#	Function Signature	Description
1	<code>group_by_length(words: List[str]) -> Dict[int, List[str]]</code>	Groups words by their length, with inner lists sorted alphabetically using <code>sorted()</code> .
2	<code>to_upper(words: List[str]) -> List[str]</code>	Returns a list of uppercase words using <code>map</code> .
3	<code>filter_short(words: List[str], max_len: int) -> List[str]</code>	Returns words with length $\leq$ <code>max_len</code> using <code>filter</code> , preserving order.
4	<code>concat_all(words: List[str]) -> str</code>	Reduces to a single string, joining words with a single space using <code>reduce</code> . Returns empty string for empty list.

#	Function Signature	Description
5	<code>zip_with_indices(words: List[str]) -> List[Tuple[int, str]]</code>	Returns <code>(index, word)</code> pairs starting at index 0 using <code>zip</code> , preserving order.
6	<code>map_lengths(words: List[str]) -> List[int]</code>	Returns a list of word lengths using <code>map</code> , preserving order.
7	<code>reduce_product_of_lengths(words: List[str]) -> int</code>	Multiplies all word lengths using <code>reduce</code> . Returns 1 for an empty list.

Test Cases

#	Input <code>words</code>	Expected Output (1-7)
1	<code>["cat", "dog", "elephant", "bee", "ant"]</code>	<code>1</code> {3: ['ant', 'bee', 'cat', 'dog'], 8: ['elephant']} <code>2</code> ['CAT', 'DOG', 'ELEPHANT', 'BEE', 'ANT'] <code>3</code> ['cat', 'dog', 'bee', 'ant'] (max_len=3) <code>4</code> "cat dog elephant bee ant" <code>5</code> [(0, 'cat'), (1, 'dog'), (2, 'elephant'), (3, 'bee'), (4, 'ant')] <code>6</code> [3, 3, 8, 3, 3] <code>7</code> 648
2	<code>[]</code>	<code>1</code> {} <code>2</code> [] <code>3</code> [] (max_len=5) <code>4</code> "" <code>5</code> [] <code>6</code> [] <code>7</code> 1
3	<code>["a", "bb", "ccc", "dd", "eeee"]</code>	<code>1</code> {1: ['a'], 2: ['bb', 'dd'], 3: ['ccc'], 4: ['eeee']} <code>2</code> ['A', 'BB', 'CCC', 'DD', 'EEEE'] <code>3</code> ['a', 'bb', 'dd'] (max_len=2) <code>4</code> "a bb ccc dd eeee" <code>5</code> [(0, 'a'), (1, 'bb'), (2, 'ccc'), (3, 'dd'), (4, 'eeee')] <code>6</code> [1, 2, 3, 2, 4] <code>7</code> 48
4	<code>["Apple", "apple", "APP"]</code>	<code>1</code> {5: ['Apple', 'apple'], 3: ['APP']} <code>2</code> ['APPLE', 'APPLE', 'APP'] <code>3</code> ['APP'] (max_len=3) <code>4</code> "Apple apple APP" <code>5</code> [(0, 'Apple'), (1, 'apple'), (2, 'APP')] <code>6</code> [5, 5, 3] <code>7</code> 75
5	<code>["αβ", "γδε", "ζ"]</code>	<code>1</code> {1: ['ζ'], 2: ['αβ'], 3: ['γδε']} <code>2</code> ['AB', 'ΓΔΕ', 'Z'] <code>3</code> ['αβ', 'ζ'] (max_len=2) <code>4</code> "αβ γδε ζ" <code>5</code> [(0, 'αβ'), (1, 'γδε'), (2, 'ζ')] <code>6</code> [2, 3, 1] <code>7</code> 6
6	<code>["", "x"]</code>	<code>1</code> {0: [''], 1: ['x']} <code>2</code> ['', 'X'] <code>3</code> ['', 'x'] (max_len=1) <code>4</code> " x" <code>5</code> [(0, ''), (1, 'x')] <code>6</code> [0, 1] <code>7</code> 0